SI2002 Formal Languages
ST0270 Formal Languages and Compilers
CM0081 Formal Languages and Automata
Project

Adolfo Andrés Castro, César Augusto Guerra, Oscar Andrés Rodriguez, Sergio Ramírez Rico

October 9, 2026

1 Logistics

This is a group assignment. Groups have two or three students; working individually is not allowed. All groups will give an oral presentation to defend their work, and all members of the group must be present.

The deadline (weeks 17–18), the submission platform and the date of the oral presentations will be defined in each class.

Academic integrity and responsible use of AI. For this project you are to read the book and implement the algorithms it describes. Every algorithm you implement must follow the book: in your report, indicate the section, algorithm or figure you followed for each part. You may use generative AI tools as a support, for example to clarify a concept, to review your text or to help you find a bug, but you are responsible for everything you submit. Do not submit code or text that you do not understand and cannot explain, and do not use AI tools to replace the reading of the book or the hand-worked examples that are required. State in your `README.md` which AI tools you used and for what purpose (or that you did not use any). During the oral presentation you may be asked to explain any part of your code, to trace an algorithm by hand on a new grammar, or to modify your code on the spot. Every member of the group must be able to do so.

2 Assignment

For this assignment, you must read Aho et al., *Compilers: Principles, Techniques, and Tools (2nd Edition)*, sections 4.4, 4.5, 4.6, 5.1 and 5.2.

The assignment consists of the following items.

1. **First and Follow.** Implement the algorithms to compute the $\textsc{First}(\cdot)$ and $\textsc{Follow}(\cdot)$ sets of a context-free grammar (Section 4.4.2 of the book). The sets must be correct for grammars with ε-productions.

2. **Parsers.** Implement the $LL(1)$ (top-down) and $SLR(1)$ (bottom-up) parsers (details below, and Sections 4.4.3–4.4.4 and 4.5–4.6 of the book).

3. **Syntax-directed definition (SDD).** Choose a grammar and define an SDD for it. Implement the evaluation of the SDD on the parse tree of an input string, using the dependency graph method (Sections 5.1 and 5.2 of the book). You may develop an interactive visualizer (optional). A step-by-step guide is given in Section 3.

4. **Report.** Document what problems you found and how you solved them, or why you could not. Submit it in PDF, preferably written in LaTeX. The required contents are given in Section 4.

5. **Oral presentation.** Slides are not required. All members of the group should be able to answer any question about the submission.

Rules for the implementation.

- The algorithms can be implemented in any programming language.
- You may use the standard library of your language, but you must not use parser generators, parsing libraries, or libraries that compute First/Follow sets, LR automata or parsing tables (for example `yacc`, `bison`, `ANTLR`, `lark`, `nltk`).
- The $LL(1)$ parser must be the table-driven (non-recursive) predictive parser of Section 4.4.4, and the $SLR(1)$ parser must be the LR parsing algorithm of Section 4.6.3. Do not transform the input grammar (for example, do not remove left recursion) before checking whether it is $LL(1)$.

2.1 Reading guide for Items 1 and 2

Take into account the following comments for the items of the assignment.

Item 1. Section 4.4.2 defines $\textsc{First}(X)$ and $\textsc{Follow}(A)$ and gives the rules to compute them. Compute $\textsc{First}(X)$ for every grammar symbol, and extend it to strings $X_1X_2\cdots X_n$ (you need it for the table). Note how ε and the end marker `$` are handled.

Your program must be able to print these sets. When it is run with the argument `--sets`, after reading the grammar it prints one line per nonterminal (in alphabetical order of the nonterminals) for each of the two sets, in the form `First(A) = {a, d}` and `Follow(A) = {b, $}`. Inside the braces, write the terminals in ASCII order, followed by `e` (for ε, if it belongs to the set) and then by `$` (if it belongs to the set). After printing the sets, the program continues as described in Section 2.2.

Item 2, $LL(1)$**.** Section 4.4.3 defines $LL(1)$ grammars and gives the algorithm that builds the predictive parsing table from $\textsc{First}(\cdot)$ and $\textsc{Follow}(\cdot)$. The condition for a grammar to be $LL(1)$ is given at the beginning of the section; implement that test in your solution. Section 4.4.4 gives the parsing algorithm that uses the table and a stack.

Item 2, $SLR(1)$**.** Section 4.5 is the background on shift-reduce parsing (read it, in particular the conflicts of Section 4.5.4). In Section 4.6.2 you find the $LR(0)$ items and the functions $\textsc{Closure}(I)$ and $\textsc{GoTo}(I,X)$, which build the canonical collection of $LR(0)$ item sets (remember to augment the grammar with a new start symbol). Section 4.6.4 builds the $\textsc{Action}(\cdot,\cdot)$ and GOTO tables from that collection and from $\textsc{Follow}(\cdot)$. The grammar is $SLR(1)$ if and only if the table has no conflicts. Section 4.6.3 gives the parsing algorithm.

2.2 Input/Output

For the input grammars, you may assume:

- The capital letter 'S' is always the initial symbol.
- Nonterminals are capital letters.
- Terminals *are not* upper-case letters. They are single characters (letters, digits or symbols), but never a blank space.
- The empty string, ε, is represented by the letter `e`. Therefore, `e` is not allowed as terminal symbol of any input grammar.
- The symbol `$` is the end marker of every string. It is not allowed as terminal symbol of any input grammar. The strings that your program receives *do not* include it: your program must append it.
- All nonterminals produce some string, that is, for every nonterminal A there exists at least one string x such that $A \xrightarrow{*} x$.
- Every nonterminal appears on the left side of exactly one input line, and the nonterminals of the grammar are exactly the n ones that appear there. The first line is not necessarily the one of `S`.

Your program should fulfill the following specifications.

2.2.1 Input

The input is a context-free grammar, as follows.

- A line with a number $n > 0$ representing the number of nonterminals of the grammar to be analyzed.
- Then, your program should read n lines with the productions given in the following format:

```
<nonterminal> -> <derivation alternatives of the nonterminal
                  separated by blank spaces>
```

  A derivation alternative is a nonempty sequence of symbols without blank spaces; the alternative ε is written `e`. For example, `B -> bBc e` has two alternatives: `bBc` and ε.

2.2.2 Output and usage

Your program must decide whether the grammar is $LL(1)$ and whether it is $SLR(1)$, and then it behaves according to the following cases. In all cases, strings to parse are read one per line, and the program stops reading strings when it receives an empty line. Because of this, the empty string is given as a line containing only `e`. For each string, print in a single line `yes` when the string is generated by the grammar and `no` otherwise (cases 1, 2 and 3).

1. The grammar is both $SLR(1)$ and $LL(1)$. Print

   ```
   Select a parser (T: for LL(1), B: for SLR(1), Q: quit):
   ```

   then read a letter, and then receive strings to parse until an empty line is given. Then, print the same query again. The program ends when `Q` is given.

2. The grammar is $LL(1)$, not $SLR(1)$. Print

   ```
   Grammar is LL(1).
   ```

   then receive strings to parse until an empty line is given, parsing with the $LL(1)$ parser.

3. The grammar is $SLR(1)$, not $LL(1)$. Print

   ```
   Grammar is SLR(1).
   ```

   then receive strings to parse until an empty line is given, parsing with the $SLR(1)$ parser.

4. The grammar is neither $LL(1)$ nor $SLR(1)$. Print

   ```
   Grammar is neither LL(1) nor SLR(1).
   ```

   and finish.

The input and output of Item 3 are described in Section 3.

3 Item 3: Guide to the SDD

In Items 1 and 2 the input and output are fixed. In Item 3, you choose the grammar, so this section describes what you must do, step by step, and what you must deliver. The idea is: *parse* a string, *build* its parse tree, and *evaluate* the attributes of the tree in an order that respects their dependencies.

Required reading. Read Sections 5.1.1 (synthesized and inherited attributes), 5.1.2 (annotated parse trees), 5.2.1 (dependency graphs), 5.2.2 (evaluation order), and 5.2.3–5.2.4 (S-attributed and L-attributed definitions). Work through the examples of these sections before you start.

Step 1: Choose the grammar

Your grammar must satisfy the following.

G1. It follows the conventions of Section 2.2 and it is $LL(1)$ or $SLR(1)$ (you must parse the strings with your own parser from Item 2).

G2. It has at least three nonterminals and at least six productions, and at least one nonterminal is recursive.

G3. It is not one of the grammars/SDDs of the book or of this document. You can take the ideas from the book, but your SDD must be different in a meaningful way (for example, other attributes or other computation). If in doubt, ask us before you start.

Step 2: Define the SDD on paper

1. Make a table of the attributes: grammar symbol, attribute name, kind (synthesized or inherited), type, and meaning. Terminals only have synthesized attributes provided by the scanner. In this project, the attribute `lexval` of a terminal is the character itself, so you do not need to write a scanner.
2. Write the semantic rules of every production in the format of the book (Fig. 5.1: a table with columns *Production* and *Semantic rules*).

Your SDD must have at least two different attributes, with at least one synthesized and at least one inherited attribute. The start symbol `S` must not have inherited attributes. Check each of the following for every production, as described in Section 5.1.1.

- Each rule defines either a synthesized attribute of the head or an inherited attribute of a symbol in the body, using only attributes of the symbols of that same production.
- For each production, every synthesized attribute of the head and every inherited attribute of each nonterminal in the body is defined exactly once.

Illustrative example (for the format only; you cannot submit it). Take the grammar `S -> P` and `P -> (P)P e`, which generates balanced parentheses. The SDD computes the maximum nesting depth `S.m`. The attribute `P.d` is inherited and is the depth at which `P` appears, and `P.m` is synthesized and is the maximum depth found in the subtree.

Production	Semantic rules
$S \to P$	$P.d = 0; \quad S.m = P.m$
$P \to (P_1)P_2$	$P_1.d = P.d + 1; \quad P_2.d = P.d; \quad P.m = \max(P_1.m, P_2.m)$
$P \to \varepsilon$	$P.m = P.d$

Step 3: Do an example by hand

Choose a string of your own with at least four terminals. Draw its parse tree and annotate every node with the values of its attributes. Draw the dependency graph of the tree (Section 5.2.1: one vertex for each attribute of each node, and one edge from each attribute that is used to the attribute that is defined by the rule). Give one valid evaluation order (a topological order, Section 5.2.2). Include this in your report.

Step 4: Classify your SDD

Decide whether your SDD is S-attributed, L-attributed, both, or neither, and justify your answer using the definitions of Sections 5.2.3 and 5.2.4. Even if it is L-attributed, you must still implement the general method of Step 5.

Step 5: Implement the evaluator

Your implementation must be organized as follows.

1. **Representation of the SDD (data, not code).** Store the SDD as data: for each production, the list of its semantic rules, and for each rule the attribute it defines, the attributes it uses, and the function that computes it. The evaluator must not know anything about your particular SDD: changing the SDD must only require changing this data.
2. **Parse tree.** Parse the input string with your parser of Item 2 (either one) and build the parse tree. Each node stores its symbol, the production used (if it is a nonterminal), its children and its attribute values.
3. **Dependency graph.** For each internal node, instantiate the rules of its production: for each rule, add one edge from each attribute it depends on to the attribute it defines. The leaves provide their `lexval`.
4. **Order and evaluation.** Compute a topological order of the graph (Section 5.2.2). If the graph has a cycle, report that the SDD is circular and stop. Otherwise evaluate the attributes in that order.
5. **Output.** For each input string, print `no` if the string is not in the language of the grammar, otherwise print
   - the annotated parse tree (one node per line, indented by depth, with its attributes),
   - the evaluation order, and
   - the value of the main attribute of the start symbol.

The grammar may be read in the format of Section 2.2; the SDD may be written in your source code. The strings are read one per line until an empty line is given.

Optional: visualizer. A text or graphical interface that shows the parse tree, the dependency graph and the evaluation step by step (for example, advancing one attribute each time the user presses a key) is welcome.

Step 6: Test

Test your evaluator with at least five strings, including the shortest string of the language, one string that is not in the language, and one string with deep nesting or recursion. In addition, show that your evaluator detects circular definitions: change one rule of your SDD temporarily so that it becomes circular, and show the output.

Common mistakes.

- Drawing the edges of the dependency graph in the wrong direction.
- Evaluating the tree recursively in a fixed order (this works only for some SDDs; your evaluator must work for any non-circular SDD).
- Forgetting that the attributes of terminals are already known, or defining an attribute twice.
- Giving inherited attributes to the start symbol.

4 Report and submission

4.1 Report (Item 4)

The PDF report must contain:

- The section(s) and algorithm(s) of the book that you followed, and the data structures that you used.
- Items 1 and 2: a grammar of your own, not in the examples of this document, with its $\textsc{First}(\cdot)$ and $\textsc{Follow}(\cdot)$ sets, and its $LL(1)$ table or its $LR(0)$ automaton and $SLR(1)$ table (computed by your program), and one string parsed step by step.

- Item 3: the grammar, the attribute table, the semantic rules, and the results of Steps 3, 4 and 6 of Section 3.
- The problems you found, how you solved them, or why you could not.

4.2 Submission

Follow these instructions.

1. The submission platform will be defined in each class.
2. A `README.md` file (Markdown format) in English is required. It must contain at least the following information:
   - Full names of group members.
   - Versions of the operating system, programming language, and tools used in your implementation.
   - Detailed instructions for building and running your implementation, including how to run Item 3.
   - The generative AI tools you used and for what purpose (or a statement that you did not use any).
3. The PDF report must be in the root directory of the submission (the LaTeX source is optional).
4. Do not include unnecessary files or directories in your submission (for example, compiled files, binaries, editor or system files).

5 Examples

These are examples, your program should run on any arbitrary context-free grammar that satisfies the conditions given in Section 2.2. An empty line ends the strings of each parser.

Example 1

Input

```
3
S -> S+T T
T -> T*F F
F -> (S) i
```

Your program should print

```
Grammar is SLR(1).
```

Then, assume it is given the strings (one at a time)

```
i+i
(i)
(i+i)*i)
```

it should print

```
yes
yes
no
```

Example 2

Input

```
3
S -> AB
A -> aA d
B -> bBc e
```

Your program should print

```
Select a parser (T: for LL(1), B: for SLR(1), Q: quit):
```

Assume `T` is given. Then, assume it is given the strings (one at a time)

```
d
adbc
a
```

it should print

```
yes
yes
no
```

Then it prints

```
Select a parser (T: for LL(1), B: for SLR(1), Q: quit):
```

Assume `Q` is given. The program ends.

Example 3

Input (this grammar is ambiguous and left-recursive)

```
1
S -> SS a
```

Your program should print

```
Grammar is neither LL(1) nor SLR(1).
```

References

Aho, Alfred V. et al. *Compilers: Principles, Techniques, and Tools (2nd Edition)*. USA: Addison-Wesley Longman Publishing Co., Inc., 2006. ISBN: 0321486811.